MirrorRoomSimulation

Документация для программиста
(актуально для текущего состояния кода)

Репозиторий: `Githu228i/MirrorRoomSimulation`

October 12, 2026

Abstract

Этот документ — карта проекта для программиста, который открывает `MirrorRoomSimulation` впервые. Здесь описано, что где лежит, как устроена архитектура, где именно находится вычислительная логика (спойлер: *не* в `backendoperators.cpp` — там пусто), как расширять код и какие в проекте есть подводные камни. Документ написан по факту чтения исходников, а не по замыслу.

Contents

1 О проекте

`MirrorRoomSimulation` — учебное/демонстрационное приложение на C++ с GUI на Qt 6 (Widgets). Пользователь кликами мыши строит замкнутую (или незамкнутую) цепочку зеркал в «комнате», редактирует точки и типы зеркал через диалоги, а затем может трассировать луч, отражающийся от зеркал.

1.1 Стек

Компонент	Назначение
C++17	Язык.
Qt 6 (Widgets)	GUI, `QPainter`, диалоги, сигналы/слоты.
CMake $\geq$ 3.16	Сборка.
Qt Creator	Рекомендуемая IDE.
`clangd`	Опционально.

1.2 Быстрый старт

```
cmake -S . -B build/Desktop-Debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/Desktop-Debug -j
./build/Desktop-Debug/MirrorRoomSimulation
```

Listing 1: Сборка

Важно. `build/`, `.qtcreator/`, `CMakeLists.txt.user`, `*.back*`, `.cache/` — не должны попадать в Git. См. **??**.

2 Структура репозитория

Файл	Назначение
`main.cpp`	`QApplication`, установка переводов, запуск `MainWindow`.
`mainwindow.h/.cpp`	Главное окно. Обработка мыши, `paintEvent`, вызов диалогов.
`mainwindow.ui`	Пустой дизайн-файл: только `centralwidget`, `menubar`, `statusbar`. Вся отрисовка — вручную в `paintEvent`.
`points.h/.cpp`	`Points` (базовая точка) и `Beam` (луч: точка + угол).
`mirror.h/.cpp`	Иерархия `Mirror` и её наследники. Пересечения зеркал. Рисование.
`polygon.h/.cpp`	`Polygon`: набор зеркал, замкнутость, валидация, трассировка луча.
`frontendoperators.h/.cpp`	Менеджер сцены: `CreatePoint`, `ConnectPoints`, `ChangeMirror`.
`backendoperators.h/.cpp`	**Пустая заглушка.** Только конструктор. Никакой логики нет.
`changeorconnectdialog.h/.cpp`	Диалог с двумя кнопками: «Изменить» / «Подключить».
`changepointdialog.h/.cpp`	Диалог редактирования точки. Определяет `ChangePointData`.

Файл	Назначение
MirrorRoomSimulation_ru_RU.ts	Файл переводов (пустой).
README.md	Одна строка.
CMakeLists.txt	Сборка.

Ориентир. Если вы ищете «где же логика backend» — идите в polygon.cpp (трассировка луча) и mirror.cpp (пересечения). backendoperators.cpp — это заготовка, которую, судя по всему, планировали наполнить, но не наполнили.

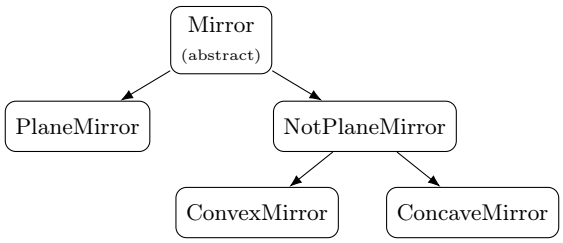
3 Архитектура

3.1 Слои

1. **Геометрия-примитив:** Points, Beam.
2. **Модель зеркал:** Mirror и наследники + геометрические алгоритмы пересечений (в анонимном namespace mirror.cpp).
3. **Композиция сцены:** Polygon — держит вектор зеркал, трассирует луч, валидирует контур.
4. **Менеджер сцены:** FrontEndOperators — обёртка над Polygon для UI-операций.
5. **GUI:** MainWindow + диалоги.

Правило. Модель (Points, Mirror, Polygon) не должна знать о QMainWindow. Однако Polygon уже знает о QPainter — наследники Mirror умеют себя рисовать. Это осознанное упрощение (см. **??**).

3.2 Иерархия классов



- Mirror — абстрактный базовый класс. Поля: start_, end_, rad_. Виртуальный деструктор обязателен — хранится в std::unique_ptr<Mirror>.
- PlaneMirror — плоское зеркало (rad = 0). Переопределяет isFlat() → true.
- NotPlaneMirror — промежуточный, защищённый конструктор.
- ConvexMirror — rad > 0, центр окружности «слева» от хорды.
- ConcaveMirror — rad < 0, центр «справа».

3.3 Поток данных

1. Пользователь кликает в MainWindow::mousePressEvent.
2. MainWindow решает: попадание в точку, близко, новая точка.
3. Вызывается один из методов FrontEndOperators: CreatePoint, ConnectPoints или ChangeMirror.
4. Тот манипулирует Polygon::sides_ (std::vector<std::unique_ptr<Mirror»).

5. `MainWindow` вызывает `update()` — перерисовка.

6. `MainWindow::paintEvent` рисует `first` и все зеркала через их виртуальный `Draw()`.

4 Ключевые сущности

4.1 Points и Beam (`points.h`)

```
class Points {
public:
    Points(double x = 0.0, double y = 0.0);
    virtual ~Points() = default;
    double getX() const;
    double getY() const;
    double Dist(const Points& other) const;
    virtual std::string Cords() const;
protected:
    double x_, y_;
};

class Beam : public Points {
public:
    Beam(double x = 0.0, double y = 0.0, double angle = 0.0);
    double getA() const;
    std::string Cords() const override;
private:
    double angle_;
};
```

Listing 2: Points / Beam

`Cords()` возвращает строку формата `"x y"` (или `"x y angle"` для луча) с точностью 3 знака. Используется только для отладочной записи в файл.

4.2 Иерархия Mirror (`mirror.h`)

Ключевые методы `Mirror`:

Метод	Назначение
`Mirror(st, end, rad)`	Бросает `std::invalid_argument` при нулевой хорде или $\|rad\| < d/2$.
`isFlat()`	`true`, если $\|rad\| < 10^{-9}$ или $> 10^{7}$.
`CircleCenter()`	Центр окружности; сторона задаётся знаком `rad`.
`CircleCenter(bool leftSide)`	Явное задание стороны.
`PointOnArc(p, c)`	Лежит ли точка на дуге/отрезке.
`Intersections(other)`	Вектор точек пересечения.
`Touches(other)`	`!Intersections().empty()`.
`Crossing(other)`	Пересекаются не только в общих концах.
`Draw(painter, insideOnLeft)`	Виртуальный. Чисто виртуальный в базе.

Знак `rad_`:

- $rad > 0$ — центр *слева* от направления `start → end`. Соответствует `ConvexMirror`.
- $rad < 0$ — центр *справа*. Соответствует `ConcaveMirror`.
- $rad = 0$ — отрезок (`PlaneMirror`).

4.3 Polygon (polygon.h)

Хранилище сцены.

```
class Polygon {
    // ...
private:
    std::vector<std::unique_ptr<Mirror>> sides_;
    Points first;              //

    bool isExist_ = false;     //
    bool isCycled_ = false;    //
    Beam beam_;                //
};
```

Listing 3: Поля Polygon

Ключевые методы:

Метод	Что делает
`Push(const Mirror&)`	Копирует зеркало через `dynamic_cast` и добавляет в `sides_`. Вызывает `Write()`.
`Del(int idx)`	Удаляет зеркало по индексу. Вызывает `Write()`.
`Check()`	Валидация: контур замкнут, стороны пересекаются только в общих вершинах соседей.
`TraceBeam(maxBounces)`	Трассировка луча. Пишет `beam.txt`.
`Write()`	Дамп вершин и луча в `points.txt`.

Побочный эффект. `Push`, `Del`, `Write`, `TraceBeam` открывают файлы. При интенсивном редактировании это может тормозить и засорять рабочую директорию.

4.4 FrontEndOperators (frontendoperators.h)

```
class FrontEndOperators {
public:
    FrontEndOperators();
    void CreatePoint(double x, double y);
    void ConnectPoints(int num);
    void ChangeMirror(int num, const ChangePointData &data);
    friend class MainWindow;
private:
    Polygon prototype;
};
```

Listing 4: FrontEndOperators

Имя вводит в заблуждение. Это *менеджер сцены*, а не слой отрисовки. Отрисовка — в `MainWindow::paintEvent` и `Mirror::Draw`.

Логика методов:

- `CreatePoint` — добавляет новую точку. Если `isExist_` ложно — сохраняет `first` и ставит флаг. Иначе строит `PlaneMirror` от конца последнего зеркала к новой точке. Лимит: 8 точек (`if (prototype.sides_.size() < 8)`).
- `ConnectPoints(num)` — замыкает контур: удаляет зеркала до `num`, добавляет последнее зеркало от `end` к `start`, ставит `isCycled_ = true`.
- `ChangeMirror(num, data)` — самая сложная функция. Разбирает случаи: последняя точка незамкнутого контура, первая точка замкнутого, обычная точка. Везде через `dynamic_cast` определяет тип зеркала и пересоздаёт соседние.

4.5 MainWindow (mainwindow.h)

Поля всего два:

```
Ui::MainWindow *ui;
FrontEndOperators polygonCreator;
```

Логика:

- `mousePressEvent` — вся обработка кликов. Хардкод: `y > 1000` отсекается, радиус попадания в точку 25, минимальное расстояние между точками 100.
- `paintEvent` — рисует `first`, линию от `first` к `temp[0]->getStart()` и все зеркала.

4.6 Диалоги

`ChangeOrConnectDialog` — простое окно с двумя кнопками (`accept` / `reject`). Текст-заглушка: `"ТУТ БУДЕТ ТВОЙ ТЕКСТ"`.

`ChangePointDialog` — редактирование одной точки. Возвращает структуру:

```
struct ChangePointData {
    double x;
    double y;
    int mirrorType;    // 0 = plane, 1 = concave, 2 = convex
    double radius;
};
```

Если `lastPoint == true` — комбобокс типа зеркала и поле радиуса отключены (у последней точки зеркала ещё нет).

4.7 BackEndOperators — пустая заглушка

```
#include "backendoperators.h"
BackEndOperators::BackEndOperators() {}
```

Listing 5: Весь файл backendoperators.cpp

> **Важно.** Не ищите здесь логику. Её нет. Вся вычислительная часть живёт в `mirror.cpp` и `polygon.cpp` в безымянных `namespace { }`. Если вы хотите что-то добавить «в backend» — логичнее либо дописать `BackEndOperators`, либо создать новый модуль, а не размазывать по `polygon.cpp`.

5 Взаимодействие пользователя

5.1 Создание точки

Клик в пустое место:

1. Если попадание в существующую точку (≤ 25) — редактирование.
2. Если близко к существующей (≤ 100) — сообщение «Точки слишком близко!».
3. Иначе — `CreatePoint(x, y)`.

Лимит: **8 сторон**.

5.2 Редактирование точки

Клик по существующей точке:

- **Контур замкнут** — сразу открывается `ChangePointDialog`.
- **Контур не замкнут** — сначала `ChangeOrConnectDialog` («Изменить» или «Подключить»).

5.3 Соединение точек

Выбор «Подключить» в незамкнутом контуре:

- Последнее зеркало подключить нельзя — `QMessageBox::critical`.
- Иначе `ConnectPoints(i)`: удаляет все зеркала до `i`, добавляет одно замыкающее.

6 Геометрические алгоритмы

6.1 Пересечения (`mirror.cpp`, анонимный namespace)

- `IntersectSegments` — отрезок/отрезок, включая коллинеарный случай с вычислением перекрытия.
- `IntersectSegmentArc` — отрезок/дуга, квадратное уравнение.
- `IntersectArcs` — дуга/дуга, стандартная формула пересечения окружностей с проверкой `PointOnArc`.

Константы: `EPS = 1e-9`, `POINT_EPS = 1e-6`, `PAR_EPS = 1e-10`, `FLAT_R = 1e7`.

6.2 Трассировка луча (`polygon.cpp`)

`TraceBeam`:

1. Позиция и направление из `beam_`.
2. Для каждого зеркала — пересечение луча.
3. Выбор ближайшего с $t > 10^{-7}$.
4. Нормаль в точке удара (`GetNormal`).
5. Отражение `d' = d - 2(d·n)n` (`ReflectDirection`), с перенормировкой.
6. Запись в `beam.txt`: позиции через пробел.

7 Файловый ввод/вывод

Файл	Кто пишет	Формат
`points.txt`	`Polygon::Write()`	Заголовок, координаты вершин, луч.
`beam.txt`	`Polygon::TraceBeam()`	x y для каждой точки удара.

Внимание. Оба файла пишутся в текущую рабочую директорию (не в `$HOME` и не в директорию проекта). При запуске из Qt Creator это `build/Desktop-Debug/`. Легко закоммитить по ошибке — держите их в `.gitignore`.

8 Сборка и CMake

```
set(PROJECT_SOURCES
    main.cpp
    mainwindow.cpp mainwindow.h mainwindow.ui
    mirror.h mirror.cpp
    points.h points.cpp
    frontendoperators.h frontendoperators.cpp
    backendoperators.h backendoperators.cpp
    polygon.h polygon.cpp
)
qt_add_executable(MirrorRoomSimulation
    ${PROJECT_SOURCES}
    changeorconnectdialog.h changeorconnectdialog.cpp
    changepointdialog.h changepointdialog.cpp
)
```

Listing 6: CMakeLists.txt, фрагмент

Странность. Диалоги добавлены отдельным блоком в `qt_add_executable`, а не в `PROJECT_SOURCES`. Если добавляете новый файл — выбирайте одно место и следуйте стилю.

8.1 Переводы

```
lupdate . -ts MirrorRoomSimulation_ru_RU.ts
lrelease MirrorRoomSimulation_ru_RU.ts
```

Баг. В `main.cpp` имя перевода — `"MirrorRoomSimulationProject_" + locale`, а CMake генерирует `MirrorRoomSimulation_`. Префикс `Project` лишний. Либо приведите `main.cpp` к `"MirrorRoomSimulation_"`, либо переименуйте `TS_FILES`. Плюс сам `.ts` пустой — переводить пока нечего.

9 Точки расширения

Задача	Куда идти
Добавить новый тип зеркала	Наследник `Mirror` + `Draw`. Плюс: конструктор в `Polygon::Push`, `ChangeMirror`, `dynamic_cast`-ветки.
Изменить физику отражения	`ReflectDirection` и `GetNormal` в `polygon.cpp` (анонимный namespace).
Добавить тип геометрического объекта	`Points` или отдельный класс.
Изменить отрисовку	`Mirror::Draw` в наследниках, `drawCircularArc` в `mirror.cpp`, `MainWindow::paintEvent`.
Добавить пункт меню / горячую клавишу	`mainwindow.ui` (пустой) или `MainWindow` в коде.
Снять лимит в 8 точек	`FrontEndOperators::CreatePoint`, условие `sides_.size() < 8`.
Настроить трассировку (число отражений)	Параметр `maxBounces` у `TraceBeam`. Сейчас не вызывается из UI.

9.1 Чек-лист при добавлении файла

1. Создать `.h` и `.cpp`.
2. Добавить в `CMakeLists.txt` (в `PROJECT_SOURCES` или рядом с диалогами — но не в оба места сразу).
3. Если класс — `QObject`, добавить `Q_OBJECT`.
4. Если есть переводимые строки — `tr(...)`.
5. Собрать, убедиться, что нет предупреждений moc/uic.
6. Закоммитить только исходники и CMake, без `build/`.

10 Известные проблемы и странности

Что	Где / почему
`BackEndOperators` пуст	`backendoperators.cpp`. Логика размазана по `polygon.cpp` и `mirror.cpp`.
Несоответствие имён переводов	`main.cpp` ищет `MirrorRoomSimulationProject_*`, CMake генерирует `MirrorRoomSimulation_*`.
Магические числа	`1600`, `1200`, `1000`, `25`, `100`, `8`, `4`. Все в `mainwindow.cpp` и `frontendoperators.cpp`.
`Polygon::Write()` на каждый `Push`/`Del`	Открывает `points.txt` при каждом изменении.
`dynamic_cast`-цепочки	`Polygon::Push`, `FrontEndOperators::ChangeMirror`. При добавлении нового типа зеркала придётся дописывать везде.
`MainWindow` — friend	У `Polygon` и `FrontEndOperators`. Ломает инкапсуляцию.
`mirror.cpp` включает `QPainter`	Модель знает о GUI. Смешение слоёв.
Текст-заглушка в диалоге	`"ТУТ БУДЕТ ТВОЙ ТЕКСТ"` в `changeorconnectdialog.cpp`.
`README.md` пуст	Одна строка.
`ChangePointData` в `changepointdialog.h`	`FrontEndOperators` тянет диалоговый заголовок только ради структуры. Логичнее вынести в отдельный `changepointdata.h`.

11 Правила работы с Git

11.1 Что не коммитить

- `build/`, `build-*/`, `cmake-build-*/`;
- `CMakeLists.txt.user`, `*.user`, `.qtcreator/`;
- `.cache/`, `compile_commands.json`;
- `*.back`, `*.back1`, `*.back2`, `* `, `*.orig`, `*.rej`;
- `points.txt`, `beam.txt` (артефакты работы).

```
build/
build-*/
cmake-build-*/
CMakeLists.txt.user
*.user
*.user.*
.qtcreator/
.cache/
compile_commands.json
*.back
*.back1
*.back2
*.back3
*~
*.orig
*.rej
points.txt
beam.txt
```

Listing 7: .gitignore

11.2 Если мусор уже в индексе

```
git rm -r --cached build/ .qtcreator/ CMakeLists.txt.user
git rm --cached points.txt beam.txt
git commit -m "Stop tracking build artifacts and IDE files"
```

11.3 Стратегия веток

Ветки `main` и `backend` имеют *разную структуру файлов*. `git merge` между ними даёт десятки конфликтов типа «modify/delete» и не имеет смысла.

- Если `backend` — актуальная версия:

```
git branch main-backup main      #
git checkout backend
git branch -f main backend
git checkout main
git push --force-with-lease origin main
```

- Если нужно перенести отдельные коммиты — `git cherry-pick <hash>`.
- Избегайте `git merge` между ветками с разной структурой файлов: Git не сопоставляет `mirror.cpp` и `backendoperators.cpp` как один модуль.

12 Соглашения по коду

- Классы — `CamelCase`; методы — `camelCase`; приватные поля — `camelCase_` (например, `sides_`, `rad_`).
- Заголовки — include guards, не `#pragma once` (в проекте везде guards).
- Не использовать `using namespace std;`.
- Forward-declaration в `.h` (`class QPainter;`) вместо `#include <QPainter>` — как в `mirror.h`.
- Публичные методы — с кратким комментарием о предусловиях (как у `Mirror`).
- Новые типы зеркал — наследовать от `Mirror`, реализовать чисто виртуальный `Draw`.

13 Отладка и типовые проблемы

Симптом	Что делать
`undefined reference to vtable`	Забыт `Q_OBJECT` (для QObject-классов) или файл не добавлен в CMake.
`moc_*.cpp` не найден	Проверить `AUTOMOC ON`.
Не рисуется новая точка	Не вызван `update()` после изменения модели.
Сцена не перерисовывается после `ChangeMirror`	Проверить возврат из `ChangeMirror` — там `return` в нескольких ветках.
`std::invalid_argument` из `Mirror`	Радиус меньше половины хорды или нулевая хорда.
Переводы не подхватываются	Префикс `MirrorRoomSimulationProject_` в `main.cpp` не совпадает с CMake. См. раздел «Переводы».
В рабочей папке появляется `points.txt`	Это `Polygon::Write()`. Добавить в `.gitignore`.
Qt Creator создаёт `*.back1/.back2`	Добавить в `.gitignore`, удалить `rm -f *.back*`.

14 Шпаргалка

```
#
cmake --build build/Desktop-Debug -j

#
rm -rf build/Desktop-Debug && \
  cmake -S . -B build/Desktop-Debug && \
  cmake --build build/Desktop-Debug -j

#                                    Qt Creator
rm -f *.back *.back1 *.back2 *.back3 *~ points.txt beam.txt

#                   ,
git ls-files | grep -E '^(build/|\.qtcreator/)' && echo "FOUND" || echo "clean"

#
lupdate . -ts MirrorRoomSimulation_ru_RU.ts
lrelease MirrorRoomSimulation_ru_RU.ts
```

Listing 8: Часто используемое

15 Куда смотреть в первую очередь

1. `main.cpp` — запуск.
2. `mainwindow.h` — состояние окна.
3. `mainwindow.cpp` — `mousePressEvent` (главный вход пользователя) и `paintEvent` (отрисовка).
4. `frontendoperators.h` — операции над сценой.
5. `frontendoperators.cpp` — `CreatePoint`, `ConnectPoints`, `ChangeMirror`.

6. `polygon.h` — модель сцены.
7. `polygon.cpp` — `TraceBeam`, отражение.
8. `mirror.h` — иерархия зеркал.
9. `mirror.cpp` — пересечения и `Draw`.
10. Диалоги — по мере необходимости.

Документ написан по факту чтения исходников. При изменении структуры обновляйте разделы 2, 3, 9, 10.